that is:

$$\frac{\partial l}{\partial \boldsymbol{W}_{1(P)}} = \frac{\partial l}{\partial \boldsymbol{W}_1} \boldsymbol{P}_C. \tag{68}$$

And we come to the attention operation, from Eq. 25, we have

$$\begin{aligned}\frac{\partial l}{\partial \boldsymbol{A}_{(P)}} &= \frac{\partial l}{\partial \boldsymbol{A}_{1(P)}} \boldsymbol{W}_{1(P)} \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}_1} \boldsymbol{W}_1 \boldsymbol{P}_C \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{P}_C,\end{aligned}$$

that is:

$$\frac{\partial l}{\partial \boldsymbol{A}_{(P)}} = \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{P}_C. \tag{69}$$

Hence we observe the permutation rules for the gradients of the intermediate-layer outputs vary from the gradients of the weights. As for the gradients of the softmax-layer output, we have

$$\begin{aligned}\frac{\partial l}{\partial \boldsymbol{S}_{(P)}} &= \frac{\partial l}{\partial \boldsymbol{A}_{(P)}} \boldsymbol{V}_{(P)}^\top \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{P}_C \cdot \boldsymbol{P}_C^\top \boldsymbol{V}^\top \boldsymbol{P}_R^\top \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{V}^\top \boldsymbol{P}_R^\top \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top,\end{aligned}$$

that is:

$$\frac{\partial l}{\partial \boldsymbol{S}_{(P)}} = \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top. \tag{70}$$

Since $\boldsymbol{S}_{(P)}$ follows Eq. 52, we have the gradients for $\boldsymbol{Q}_{(P)}$ combining with Eq. 70:

$$\begin{aligned}\frac{\partial l}{\partial \boldsymbol{Q}_{(P)}} &= \frac{1}{\sqrt{d}}[(diag(\boldsymbol{S}_{(P)}) - \boldsymbol{S}_{(P)}^\top \boldsymbol{S}_{(P)}) \frac{\partial l}{\partial \boldsymbol{S}_{(P)}}] \boldsymbol{K}_{(P)} \\ &= \frac{1}{\sqrt{d}}[(\boldsymbol{P}_R diag(\boldsymbol{S}) \boldsymbol{P}_R^\top - \boldsymbol{P}_R \boldsymbol{S}^\top \boldsymbol{P}_R^\top \cdot \boldsymbol{P}_R \boldsymbol{S} \boldsymbol{P}_R^\top) \\ &\quad \cdot \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top] \boldsymbol{P}_R \boldsymbol{K} \boldsymbol{P}_C^\top \\ &= \frac{1}{\sqrt{d}}[(\boldsymbol{P}_R diag(\boldsymbol{S}) \boldsymbol{P}_R^\top - \boldsymbol{P}_R \boldsymbol{S}^\top \boldsymbol{S} \boldsymbol{P}_R^\top) \\ &\quad \cdot \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top] \boldsymbol{P}_R \boldsymbol{K} \boldsymbol{P}_C^\top \\ &= \frac{1}{\sqrt{d}}[\boldsymbol{P}_R (diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S}) \\ &\quad \cdot \boldsymbol{P}_R^\top \cdot \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top] \boldsymbol{P}_R \boldsymbol{K} \boldsymbol{P}_C \\ &= \frac{1}{\sqrt{d}}[\boldsymbol{P}_R (diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S}) \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top] \boldsymbol{P}_R \boldsymbol{K} \boldsymbol{P}_C \\ &= \frac{1}{\sqrt{d}} \boldsymbol{P}_R (diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S}) \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{P}_R^\top \cdot \boldsymbol{P}_R \boldsymbol{K} \boldsymbol{P}_C \\ &= \boldsymbol{P}_R \frac{1}{\sqrt{d}} (diag(\boldsymbol{S}) - \boldsymbol{S}^\top \boldsymbol{S}) \frac{\partial l}{\partial \boldsymbol{S}} \boldsymbol{K} \boldsymbol{P}_C \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{Q}} \boldsymbol{P}_C.\end{aligned}$$

By a similar derivation on $\boldsymbol{K}$ we obtain:

$$\frac{\partial l}{\partial \boldsymbol{K}_{(\boldsymbol{P})}} = \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{K}} \boldsymbol{P}_C. \tag{71}$$

Following a similar proof to the gradients of $\boldsymbol{W}_{1(P)}$ or $\boldsymbol{W}_{2(P)}$, we could easily derive:

$$\frac{\partial l}{\partial \boldsymbol{W}_{Q(P)}} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{W}_Q} \boldsymbol{P}_C, \tag{72}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_{K(P)}} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{W}_K} \boldsymbol{P}_C. \tag{73}$$

By Eq. 28, the gradient of $\boldsymbol{V}_{(P)}$ is

$$\begin{aligned}\frac{\partial l}{\partial \boldsymbol{V}_{(P)}} &= \boldsymbol{S}_{(P)}^\top \frac{\partial l}{\partial \boldsymbol{A}_{(P)}} \\ &= \boldsymbol{P}_R \boldsymbol{S} \boldsymbol{P}_R^\top \cdot \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{A}} \boldsymbol{P}_C \\ &= \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{V}} \boldsymbol{P}_C,\end{aligned}$$

and thus we have

$$\frac{\partial l}{\partial \boldsymbol{V}_{(P)}} = \boldsymbol{P}_R \frac{\partial l}{\partial \boldsymbol{V}} \boldsymbol{P}_C, \tag{74}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_{V(P)}} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{W}_V} \boldsymbol{P}_C. \tag{75}$$

So far, we have proved the rule for the gradient of weight matrices:

$$\frac{\partial l}{\partial \boldsymbol{W}_{i(P)}} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{W}_i} \boldsymbol{P}_C, \quad i \in \{Q, K, V\}. \tag{76}$$

$$\frac{\partial l}{\partial \boldsymbol{W}_{1(P)}} = \frac{\partial l}{\partial \boldsymbol{W}_1} \boldsymbol{P}_C, \quad \frac{\partial l}{\partial \boldsymbol{W}_{2(P)}} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{W}_2}. \tag{77}$$

$\boldsymbol{W}_{i(P)}$ are the weights of $\mathrm{Enc}_{(P)}$ while $\boldsymbol{W}_i$ are the weights of Enc. By induction, we can reach the conclusion that if a Transformer encoder block is randomly initialized and trained with $\boldsymbol{Z}_{(P)}$, it would eventually learn to become $\mathrm{Enc}_{(P)}$, the weights of which are associated with Enc by Eq. 76 and Eq. 77. The proof of backward equivariance on the linear projection in the attention is omitted as its proof is similar and the conclusion is the same with Eq. 76. Hence we have proved backward permutation in-/equi-variance. □

### 10.3. Proofs on Embeddings

We show in this section that the parameters of the embedding layer $F_1$, including the position embeddings, are the same despite Alg. 1 is applied or not.

**Theorem 10.1.** *The parameters of $F_1$ trained with or without permutation are the same.*

*Proof.* We denote the output of $F_1$ as $\boldsymbol{Z}_0$, and the input of the Transformer backbone as $\boldsymbol{Z}$. In normal setting (Eq. 1), $\boldsymbol{Z}$ equals to $\boldsymbol{Z}_0$, and so do their gradients. In the permuted setting where we use subscript $_{(P)}$ to denote all the varibles, the input to the backbone is the permuted output of $F_{1(P)}$:

$$\boldsymbol{Z}_{(P)} = \boldsymbol{P}_R \boldsymbol{Z}_{0(P)} \boldsymbol{P}_C. \tag{78}$$

To prove the weights of $F_{1(P)}$ is equivalent to those of $F_1$, we need to prove:

$$\frac{\partial l}{\partial \boldsymbol{Z}_{0(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}_0}. \tag{79}$$

It is clear that

$$\begin{aligned}
\mathrm{d}l &= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{Z}_{(P)}}^\top \mathrm{d}\boldsymbol{Z}_{(P)}) \\
&= \mathrm{tr}(\boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{Z}}^\top \boldsymbol{P}_R^\top \boldsymbol{P}_R \mathrm{d}\boldsymbol{Z}_{0(P)} \boldsymbol{P}_C) \\
&= \mathrm{tr}(\boldsymbol{P}_C \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{Z}}^\top \mathrm{d}\boldsymbol{Z}_{0(P)}) \\
&= \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{Z}}^\top \mathrm{d}\boldsymbol{Z}_{0(P)}),
\end{aligned}$$

where the second equality holds by Lemma 4.8. Hence,

$$\frac{\partial l}{\partial \boldsymbol{Z}_{0(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}} = \frac{\partial l}{\partial \boldsymbol{Z}_0}. \tag{80}$$

The invariance of the weights of $F_1$ can be derived from Eq. 80. □

It is worth noting that the position embeddings are added before the permutation, so Transformer gets the right position information, since Transformer modeling the position by the position embeddings instead of the order of the input.

### 10.4. Proofs on Masked Attention

Masked attention is an essential component in the Transformer Decoder Block which is the backbone of the generetive language model [3, 27]. The token permutation can hardly pass through the nonlinear effect of the masked attention, but the column permutation cancels out before the mask takes effect. Thus for the masked attention, we apply column permutations only.

**Theorem 10.2.** *The results of Masked Softmax with or without column permutation are the same:*

$$MS_{(P)} = MS. \tag{81}$$

*Proof.* For $MS = Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}})$:

$$\begin{aligned}
MS_{(P)} &= Masked - Softmax(\frac{\boldsymbol{Q}_{(P)}\boldsymbol{K}_{(P)}^\top}{\sqrt{d}}) && (82)\\
&= Masked - Softmax(\frac{\boldsymbol{Q}\boldsymbol{P}_C \cdot \boldsymbol{P}_C^\top \boldsymbol{K}^\top}{\sqrt{d}}) && (83)\\
&= Masked - Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}) && (84)\\
&= Masked - Softmax(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}) && (85)\\
&= MS. && (86)
\end{aligned}$$

□

## 11. Detailed Experimental Setup

### 11.1. Training Setup

In fine-tuning ViT-Base on Cifar10, an Adam optimizer with a fixed learning rate $10^{-4}$ is used. The model is trained for 5 epochs with a cross-entropy loss. In fine-tuning Bert and GPT2 on IMDB classification, an Adam optimizer with a fixed learning rate $10^{-5}$ is used. The models are trained for 2 epochs.

For the 'unauthorized' and train-from-scratch setting on Cifar10 of ViT, the default training setting of ViT is used. Random data augmentation and cosine scheduler are added and the model is trained for 200 epochs till convergence.

The text generation in the properties validation experiments is zero-shot learning on huggingface pre-trained GPT2. The model is fine-tuned with Adam optimizer and learning rate $10^{-5}$ for 1 epoch.

For the CelebA attribute classification task in the 'privacy-preserving split learning' experiments, we adopt `timm` pre-trained model vit_base_patch16_224 (ViT-Base) to transfer to a 40-binary-attribute classification task. A SGD optimizer is used with a cosine scheduler, for which the (initial, final) learning rate are set to $(0.05, 2 \times 10^{-4})$ and $(5 \times 10^{-4}, 2 \times 10^{-6})$ for the classification head and the encoder blocks, respectively.

## 11.2. Threat Model of Privacy-Preserving Split Learning

The threat model is consist with [15, 33]. We assume the edge holds a private training data set $\mathbb{D}_{train} = \{X, Y\}$, where $X$ are the private data and $Y$ are the private labels. The edge aims at training a model with the assistance of the cloud, yet without exposing the private data. The cloud possesses powerful computing power and is honest-but-curious, meaning it obeys the protocol and performs the learning task accordingly, but is curious about the private data. The edge selects a model and splits it into three parts: $F_1, \mathrm{Enc}, F_2$. $F_1, F_2$ are parts close to the input layer and the output layer, respectively, which are sufficiently lightweight to deploy on the edge, whereas $\mathrm{Enc}$ is the major part run in the cloud.

Referring to the loss function as $L_{task}$ and the local privacy-preserving method as $M$, the ultimate goal is to jointly train $F_1, \mathrm{Enc}, F_2$ to

$$\underset{F_1, \mathrm{Enc}, F_2}{\text{minimize}} \; L_{task}(F_2(\mathrm{Enc}(F_1(X))), Y), \tag{87}$$

without the edge revealing $X, Y$ to the cloud. Accessing $F_1(X)$ should not permit the cloud to infer about $X$. The cloud could launch black-box inversion attacks:

**Black-box attackers** collect the auxiliary data set $X_{aux}$ and the corresponding features under protection mechanism $M$ as $M(F_1(X_{aux}))$, maybe over multiple training rounds. The attacker trains an inversion model $G$ over $(X_{aux}, M(F_1(X_{aux})))$ to invert the raw input from features [7, 10, 24] by

$$\underset{G}{\text{minimize}} \; L_{atk}(G(M(F_1(X_{aux}))), X_{aux}). \tag{88}$$

The loss $L_{atk}$ can be the mean square error (MSE) between the reconstructed input $\tilde{X}_{aux}$ and $X_{aux}$. At convergence, $G$ works as a decoder to invert features into inputs. We adopt the MAE decoder $G$ with base-size Transformer backbone and a Tanh activation layer, pre-trained on ImageNet, as the model inversion model. We train $G$ with an AdamW optimizer with a learning rate of $10^{-4}$ for 30 epochs.